

Routing

Multi-Page Navigation in Single-Page Apps

■ LEARNING OUTCOME

By the end of this module you will build multi-page React applications with React Router v6, implement dynamic routes, protect routes, and handle navigation programmatically.

■ Skills You Will Gain

- Set up React Router v6
- Create routes with nested routes
- Navigate with Link, NavLink, useNavigate
- Read URL params and query strings
- Implement protected routes
- Use lazy loading for code splitting

■ **React Router is the standard routing solution for React. Used in production by millions of apps, it enables navigation that feels instant while managing the browser URL and history.**

SECTION 1

React Router v6 Setup

```
npm install react-router-dom // src/App.jsx import { Routes, Route, Navigate } from
'react-router-dom'; import { lazy, Suspense } from 'react'; const Home = lazy(() =>
import('./pages/Home')); const Products = lazy(() => import('./pages/Products')); const
ProductDetail = lazy(() => import('./pages/ProductDetail')); const Dashboard = lazy(() =>
import('./pages/Dashboard')); const NotFound = lazy(() => import('./pages/NotFound')); function
App() { return ( <Suspense fallback={<div>Loading...</div>}> <Routes> <Route path='/'
element={<Home />} /> <Route path='/products' element={<Products />} /> <Route path='/products/:id'
element={<ProductDetail />} /> { /* Nested routes */} <Route path='/dashboard'
element={<ProtectedRoute><DashboardLayout /></ProtectedRoute>} <Route index
element={<DashboardHome />} /> <Route path='orders' element={<Orders />} /> <Route path='settings'
element={<Settings />} /> </Route> <Route path='/old' element={<Navigate to='/new' replace />} />
<Route path='*' element={<NotFound />} /> </Routes> </Suspense> ); } // Wrap in BrowserRouter in
main.jsx import { BrowserRouter } from 'react-router-dom'; <BrowserRouter><App /></BrowserRouter>
```

SECTION 2

Navigation Hooks and Protected Routes

```
import { Link, NavLink, useNavigate, useParams, useSearchParams, useLocation, Outlet } from
'react-router-dom'; // Link -- no page reload <Link to='/products'>Products</Link> <NavLink
to='/dashboard' className={({ isActive }) => isActive ? 'nav-link active' : 'nav-link'}> Dashboard
</NavLink> // useNavigate -- programmatic navigation function LoginForm() { const navigate =
useNavigate(); const handleLogin = async (creds) => { await loginUser(creds);
navigate('/dashboard', { replace: true }); // no back button }; return <form
onSubmit={handleLogin}>...</form>; } // useParams -- URL parameters function ProductDetail() {
const { id } = useParams(); // from path='/products/:id' const { data } =
useFetch(`/api/products/${id}`); return <div>{data?.name}</div>; } // useSearchParams -- query
string (?page=1&sort=price) function ProductList() { const [sp, setSP] = useSearchParams(); const
page = parseInt(sp.get('page') || '1'); const sort = sp.get('sort') || 'name'; return <select
value={sort} onChange={e => setSP({ page:1, sort:e.target.value })}> <option
value='name'>Name</option><option value='price'>Price</option> </select>; } // Protected route
function ProtectedRoute({ children }) { const { user, isLoading } = useAuth(); const location =
useLocation(); if (isLoading) return <Spinner />; if (!user) return <Navigate to='/login' state={{
from: location }} replace />; return children; } // Outlet -- renders nested child routes function
DashboardLayout() { return <div className='dashboard'><Sidebar /><main><Outlet /></main></div>; }
```

■ PRACTICE Q & A

Q1. What is the difference between Link and a in React Router?

A: *<a> causes full page reload. Link uses History API for instant client-side navigation -- no reload, preserves state.*

Q2. What does replace do in navigate()?

A: *Replaces current history entry -- user cannot go back. Use for post-login redirects or after form submission.*

Q3. What is the index route?

A: *Route with 'index' attribute renders at parent path exactly. <Route index element={<DashboardHome />}/> renders at /dashboard.*

Q4. How does lazy loading work with React Router?

A: *lazy(() => import('./Page')) loads code only when first rendered. Suspense shows fallback while loading. Route bundle downloads only when visited.*

Q5. What is useSearchParams?

A: Stateful way to read/update URL query string. setSearchParams updates URL without reload, letting users bookmark filtered views.

■■ Routing Cheat Sheet	
<code><BrowserRouter></code>	Wrap app for routing context
<code><Route path="/" element={<Home/>}/></code>	Define a route
<code><Route path="/p/:id" element={<D/>}/></code>	URL parameter route
<code><Route index element={<H/>}/></code>	Default nested route
<code><Navigate to="/login" replace/></code>	Redirect element
<code><Link to="/page">Text</Link></code>	Client navigation -- no reload
<code><NavLink className={({isActive})=>...}></code>	Active-aware nav link
<code>useNavigate()</code>	Programmatic navigation hook
<code>navigate('/path',{replace:true})</code>	Navigate without history
<code>useParams()</code>	Read :param from URL
<code>useSearchParams()</code>	Read/write query string
<code><Outlet/></code>	Render nested route